

Tecnologias de Comunicação para Sistemas Críticos

Laboratory Class PL2 26-27 edition – 3 hours

TCP protocols

(Exercises adapted from RCOMP 21/22)

Table of Contents

1. Using TCP with Berkeley Sockets – C Language.....	2
1.1. TCP connection establishment.....	2
1.2. Sending and receiving data (reading and writing).....	3
1.2.1 A simple TCP client-server example.....	3
1.2.2 challenge.....	7
1.3 Multi-process TCP servers.....	8
1.4. Port numbers and pending requests queue.....	9
2. Implementing a Multi-process TCP client and server.....	10
2.1. [Lab02.ex02] The TCP client in C language (tcp_cli_sum.c).....	10
2.2. [Lab02.ex02] TCP server in C language (tcp_srv_sum.c) – Linux Multi-Process	12
3. [Lab02.ex03] Implementing a Key-Value store.....	15
3.1. [Lab02.ex03] LLM challenge.....	15

TCP is a reliable connection-oriented transport protocol with automatic error correction. It establishes a dedicated communication channel (TCP connection) between a pair of applications. Through the TCP connection, data is sent in a continuous byte stream, preserving the data sequence, and guaranteeing data delivery.

Because it is connection-oriented, prior to data transactions, a connection between two applications must be established.

One of the two enrolled applications must take the initiative of emitting a connection request to the counterpart. In a client-server architecture, that role is therefore taken by the client. Once the TCP connection is established between the two applications, data can be sent and received simply by writing and reading bytes, however, these operations must be synchronized at the byte level.

This is rather different from UDP, where synchronization is at datagram level. In UDP, a datagram sending operation in one application must match a datagram receiving operation in the counterpart. However, the application receiving a datagram does not require to know how many bytes it will be receiving, just that is receiving a datagram, the number of bytes transported by the datagram will be known at its reception.

With TCP, a byte level synchronization is required, this means the writing (sending) of N bytes in one application, must be matched by reading (receiving) the same N bytes in the counterpart application.

If the number of bytes we try to read is less than those written, some will be unread and will appear in the next reading operation. If the number of bytes we try to read is greater than those written, then, the reading operation will block waiting for the missing bytes.

In TCP, synchronization is a key feature to be settled by the **application protocol**, for the required byte level synchronization on TCP connections, three approaches can be used:

- a) Use a pre-agreed fixed number of bytes in each transaction, this way the reader always knows how many bytes it should read.
- b) Before sending the data itself, send information about the number of bytes the data is made of. The reader starts by getting the data length, then it knows how many data bytes it should read next. This solution is used in HTTP protocol, where the message header has a Content-Length field indicating the number of bytes in the message's body.
- c) Use a specific pre-agreed byte (or bytes sequence) as an end-of-data marker. Thus, the receiver must then read one byte at a time, and check if it's the end-of-data marker. This solution is also used in HTTP, the CR+LF sequence is used to mark the end of each header's line, also, the CR+LF+CR+LF sequence (an empty line) is used to mark the header's end.

This last alternative is easy to implement if data is made of a limited set of possible byte values, like with ASCII text. If data bytes are allowed to have any value, additional processing will be required, namely, any mark value occurring in data will have to be masked on the sender and unmasked on the receiver to avoid being wrongly interpreted.

1. Using TCP with Berkeley Sockets – C Language

1.1. TCP connection establishment

As mentioned before, to establish a TCP connection two applications must assume different roles, therefore each will use a different function to perform its specific role.

int connect(int socket, struct sockaddr *address, int address_len);

connect() – is called by the application wishing to create the TCP connection (the TCP client), for successful completion, this must match an **accept()** function call on the counterpart. The connect() function receives as argument a pointer to a caller-defined structure with the server's IP address and port number, to where the connection request will be sent. If an error occurs it returns -1, otherwise the TCP connection is established, and the socket is then connected to the counterpart.

int accept(int socket, struct sockaddr *address, int *addrlen);

accept() – is called by the application that receives a TCP connection request (the TCP server), this is a **blocking function**, when called, if there are no pending connection requests it will block the application execution and wait until it receives one. If **address** is not NULL it will be used to store the counterpart's IP address and port number, if so the value in **addrlen** must be defined by the caller otherwise it can also be NULL.

When the `accept()` function unblocks with no error (on error it returns -1), the TCP connection is established and **a new socket is returned**. The new socket, returned by `accept()`, is connected to the counterpart. The original socket, used in the `accept()` function call, is kept **open** and available for receiving other TCP connection requests from other clients.

1.2. Sending and receiving data (reading and writing)

Once the TCP connection is established, sending and receiving data can be accomplished by using the standard **write()** and **read()** functions over the connected socket. Data is sent and received in a continuous byte stream.

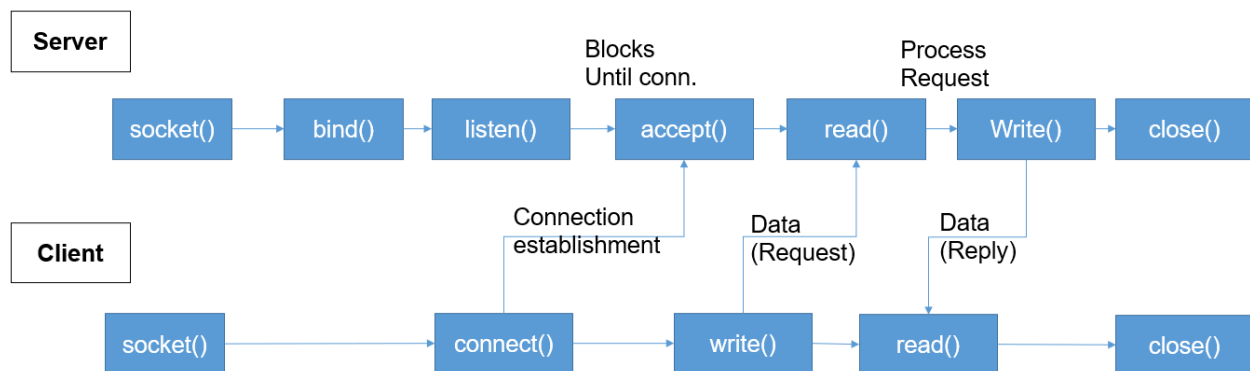


Figure 1 TCP flow

1.2.1 A simple TCP client-server example

One of the differences between TCP and UDP is that UDP is **connection-less**. In other words, the connection between client and server does not need to be kept live for the applications to work.

In the TCP protocol what happens when a new client tries to connect with the server, when the server has already established a connection with another client (that is still active)?

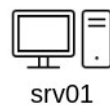
Using the following client-server example, recreate the scenario and describe what happened.

For this, **execute a server in one machine, and two clients in two separate machines.**

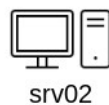
If using the virtual machines created on the previous class, clone one of the servers, change the **hostname** of the new server, and map it for the other servers. See Figure 2.

[Lab02.ex01] TCP clients and servers

user "root" with "toor"
password
cd /mnt/shared



srv01



srv02



srv03

Clone one of the virtual machines, name it: server 03, ex:

```
nano /etc/hostname
```

```
srv03
```

```
nano /etc/hosts
```

```
127.0.0.1      srv03
```

```
192.168.56.101 srv01
```

```
192.168.56.102 srv02
```

Adapt to your IP's and restart the machine

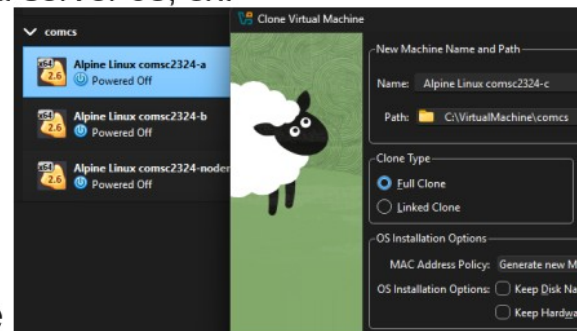


Figure 2 Three servers for first exercise

[Lab02.ex01] The TCP client in C language (tcp_cli_sum.c)

```
#include <stdio.h>
#include <string.h> // bzero()
#include <stdlib.h> // exit()
#include <sys/socket.h>
#include <arpa/inet.h> // inet_addr()
#include <unistd.h> // read(), write(), close()

int main(int argc, char **argv) {
    int sockfd;
    struct sockaddr_in server_addr;
    char server_message[2000], client_message[2000];

    if(argc!=2) {
        puts("Server's IPv4 name is required as argument");
        exit(1);
    }
```

```

}

// Clean buffers:
bzero(server_message, sizeof(server_message));
bzero(client_message, sizeof(client_message));

// Create socket:
sockfd = socket(AF_INET, SOCK_STREAM, 0);

if(sockfd < 0){
    printf("Unable to create socket\n");
    return -1;
}

printf("Socket created successfully\n");

// Set port and IP the same as server-side:
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(2000);
server_addr.sin_addr.s_addr = inet_addr(argv[1]);

// Send connection request to server:
if(connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0){
    printf("Unable to connect\n");
    close(sockfd);
    return -1;
}
printf("Connected with server successfully\n");

// Get input from the user:
printf("Enter message: \n");
fgets(client_message, sizeof(client_message), stdin);

// Send the message to server:
if( write(sockfd, client_message, sizeof(client_message)) < 0){
    printf("Unable to send message\n");
    close(sockfd);
    return -1;
}

// Receive the server's response:
if( read(sockfd, server_message, sizeof(server_message)) < 0){
    printf("Error while receiving server's msg\n");
    close(sockfd);
    return -1;
}

printf("Server's response: %s\n", server_message);

// Close the socket:
close(sockfd);

return 0;
}

```

[Lab02.ex01] The TCP **Server** in C language (tcp_srv_sum.c)

```
#include <stdio.h>
#include <string.h> // bzero()
#include <sys/socket.h>
#include <arpa/inet.h> // inet_addr()
#include <unistd.h> // read(), write(), close()

int main()
{
    int sockfd, client_sock;
    struct sockaddr_in server_addr, client_addr;
    char server_message[2000], client_message[2000];

    // Clean buffers:
    bzero(server_message, sizeof(server_message));
    bzero(client_message, sizeof(client_message));

    // Create socket:
    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if(sockfd < 0){
        printf("Error while creating socket\n");
        return -1;
    }
    printf("Socket created successfully\n");

    // Set port and IP:
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(2000);
    server_addr.sin_addr.s_addr = inet_addr("0.0.0.0");

    // Bind to the set port and IP:
    if(bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr))<0){
        printf("Couldn't bind to the port\n");
        close(sockfd);
        return -1;
    }
    printf("Done with binding\n");

    // Listen for clients:
    if(listen(sockfd, 1) < 0){
        printf("Error while listening\n");
        close(sockfd);
        return -1;
    }
    printf("\nListening for incoming connections.....\n");

    // Accept an incoming connection:
    socklen_t slen=sizeof(client_addr);
    client_sock = accept(sockfd, (struct sockaddr*)&client_addr, &slen);

    if (client_sock < 0){
        printf("Can't accept\n");
        close(sockfd);
        return -1;
    }
}
```

```

    printf("Client connected at IP: %s and port: %i\n", inet_ntoa(client_addr.sin_addr),
    ntohs(client_addr.sin_port));

    // Receive client's message:
    if( read(client_sock, client_message, sizeof(client_message)) < 0){
        printf("Can't read\n");
        return -1;
    }

    printf("Msg from client: %s\n", client_message);

    // Respond to client:
    strcpy(server_message, "This is the server's message.");

    printf("Processing message from client...\n");
    sleep(10); //this will hold the server to reply the client's message for 10 seconds

    if( write(client_sock, server_message, sizeof(server_message)) < 0){
        printf("Can't write\n");
        return -1;
    }

    // Closing the socket:
    close(client_sock);
    close(sockfd);

    return 0;
}

```

1.2.2 challenge

In the TCP protocol what happens when a new client tries to connect with the server, when the server has already established a connection with another client (that is still active)?

On the server side, the sleep() function is used to simulate the processing of the message holding the server to reply the client for 10 seconds :

...

```

    printf("Processing message from client...\n");
    sleep(10);

```

...

Put here the answer:

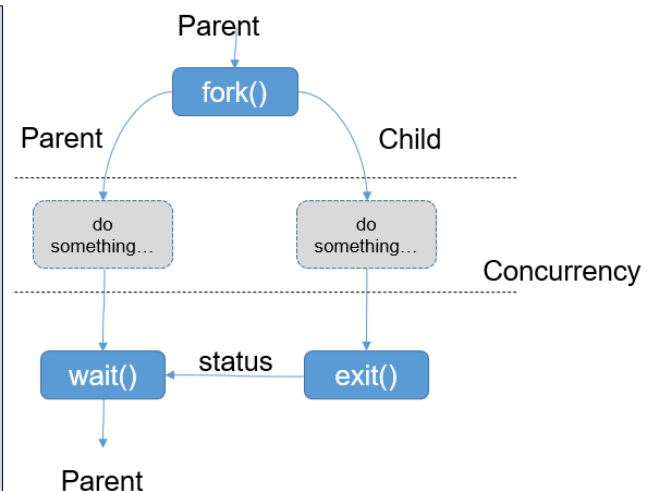
1.3 Multi-process TCP servers

A TCP server job is complex because it must be always available to accept a new incoming connections requests, while, at the same time it must read incoming messages from already connected clients, in every connected socket. Each time the **accept()** function returns with success, there is one more connected socket for the server application to handle with. One way **to solve this issue is by creating a parallel sub-task** (e.g., process or thread) for each socket.

In the following typical implementation layout, the parent process keeps calling the **accept()** function. For each established connection, it creates a child process with the purpose of handling the connected client requests on the new socket.

fork() system call is used for **creating a new process**, which is called **child process**, and child(s) **runs concurrently** with the process that makes the fork() call (**parent process**).

After a new child process is created, **both processes will execute the next instruction** following the fork() system call. A child process uses the same pc(program counter), same CPU registers, same open files which use in the parent



```
for(;;){  
  
    newSock=accept(sock, ...);  
  
    if(!fork()) {  
        // child process
```

```

        close(sock);
        /* process all client requests on newSock */
        ...
        close(newSock);
        exit(0); // child exits
    }
    close(newSock); // parent process
}

```

Using this type of solution has some advantages, each client has an independent process dedicated to it, thus interference's between sessions of different clients are unlikely. However, if some type of interaction between clients' sessions is required, then IPC (Inter Process Communication) will have to be used.

1.4. Port numbers and pending requests queue

As usual for clients, a **TCP client doesn't need a fixed local port number**, in fact, for TCP clients, binding is optional. When the **connect()** function is called if the socket is not bound, then it is bounded to one local free port number.

Again, as usual for servers, the TCP server **must use a fixed local port number** so that clients know to where they should send the connection request.

Additionally, after binding to a fixed local port number, the TCP server must also set the size of the pending connections requests queue (pending stands for not yet accepted), the maximum possible size is defined by **SOMAXCONN**.

A code example for a TCP server socket setup is:

```

bind(sock, ...);
listen(sock,SOMAXCONN);
newSock=accept(sock, ...);

```

2. Implementing a Multi-process TCP client and server

Create a TCP client and a TCP server with the following applications features and application protocol specification:

- The server IP address (IPv4, IPv6 or DNS name) is provided to the client as the first argument at the command line.
- Once connected, the client sends a list of integer numbers terminated with the zero value.
- The server accepts TCP connections from clients. For logging, each new connection and disconnection should be presented at the server console, showing the client IP address and port number.
- The server calculates the sum of the sent integers and sends back the result.
- When the client wants to exit it should send an empty list (started by the zero value).
- Each integer is sent as a sequence of 4 bytes in order of increasing significance, i.e., first the LSB (Least Significant Byte) and last the MSB (Most Significant Byte).

So, the sequence of bytes A, B, C, D represents the number given by:

$$\text{NUMBER} = A + 256 \times B + 256 \times 256 \times C + 256 \times 256 \times 256 \times D$$

For instance:

The number 10 is sent as the sequence of bytes: 10, 0, 0, 0

The number 300 is sent as the sequence of bytes: 44, 1, 0, 0

This might look an odd way of sending an integer number. The problem is, directly sending integer numbers as they are stored in the local host's memory is not an option because they can be stored differently in the source host and destination host, for instance when a client implemented in a different language is sending to a C server.

Another used option to send data is by using an implementation independent representation, i.e., sending humanly readable representation of data. This application protocol could specify that each integer is sent in the form of its text representation.

This is a good option because all programming APIs have functions to parse most data types from textual representations into local storing and functions to produce textual representations from local storing, the only issue is that such solutions require more processing power.

2.1. [Lab02.ex02] The TCP **client** in C language (tcp_cli_sum.c)

```
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
```

```

#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUF_SIZE 30
#define SERVER_PORT "9999"

// read a string from stdin protecting buffer overflow
#define GETS(B,S) {fgets(B,S-2,stdin);B[strlen(B)-1]=0;}

int main(int argc, char **argv) {
    int err, sock;
    unsigned long f, i, n, num;
    unsigned char bt;
    char line[BUF_SIZE];
    struct addrinfo req, *list;

    if(argc!=2) {
        puts("Server's IPv4/IPv6 address or DNS name is required as argument");
        exit(1);
    }

    bzero((char *)&req,sizeof(req)); // let getaddrinfo set the family depending on the
    supplied server address
    req.ai_family = AF_UNSPEC;
    req.ai_socktype = SOCK_STREAM;
    err=getaddrinfo(argv[1], SERVER_PORT , &req, &list);

    if(err) {
        printf("Failed to get server address, error: %s\n",gai_strerror(err));
        exit(1);
    }

    sock=socket(list->ai_family,list->ai_socktype,list->ai_protocol);

    if(sock==-1) {
        perror("Failed to open socket");
        freeaddrinfo(list);
        exit(1);
    }
    if(connect(sock,(struct sockaddr *)list->ai_addr, list->ai_addrlen)==-1) {
        perror("Failed connect");
        freeaddrinfo(list);
        close(sock);
        exit(1);
    }

    do {
        do {
            printf("Enter a positive integer to SUM (zero to terminate): \n");
            GETS(line,BUF_SIZE);
            while(sscanf(line,"%li",&num)!=1 || (long int)num<0) {
                puts("Invalid number");
                GETS(line,BUF_SIZE);
            }
        }
    }

```

```

        n=num;
        for(i=0;i<4;i++) {
            bt=n%256;
            write(sock,&bt,1);
            n=n/256;
        }
    } while(num);
    num=0;
    f=1;
    for(i=0;i<4;i++) {
        read(sock,&bt,1);
        num=num +bt*f; f=f*256;
    }
    printf("SUM RESULT=%lu\n",num);
} while(num);
close(sock);
exit(0);
}

```

To create the appropriate socket, the same tactic as with previous UDP clients is used: let **getaddrinfo()** determine the address family of the provided server address, and then, create the local socket accordingly.

Nevertheless, other strategies are possible to support both IPv4 and IPv6 addresses, one would be, using always an IPv6 socket. The issue when using an IPv6 socket is it can't handle an IPv4 server address. The solution would require determining if the server's address is IPv4, and in that case transform the IPv4 address into IPv4-Mapped (A.B.C.D -> ::ffff:A.B.C.D).

Back to the sample code, the SOCK_STREAM (TCP) socket is then created using the values provided by **getaddrinfo()** and the TCP connection is established to the server node IP address and port number. If the connection establishment fails (for instance because the server is not running), then **connect()** returns -1.

For each user-entered integer number, the four bytes representing it are sent to the server, when the number sent is zero (end of the list), the server is supposed to send back a reply, also as an **integer** in the form of **4 bytes**.

If the reply is zero, this means we have sent an empty list and want to exit, so we close the socket, and thus the TCP connection.

2.2. [Lab02.ex02] TCP **server** in C language (tcp_srv_sum.c) – **Linux Multi-Process**

```

#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <sys/types.h>

```

```

#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUF_SIZE 300
#define SERVER_PORT "9999"

int main(void) {
    struct sockaddr_storage from;
    int err, newSock, sock;
    unsigned int adl;
    unsigned long i, f, n, num, sum;
    unsigned char bt;

    char cliIPtext[BUF_SIZE], cliPortText[BUF_SIZE];
    struct addrinfo req, *list;
    bzero((char *)&req, sizeof(req));

    // requesting an IPv4 type of communication
    req.ai_family = AF_INET6;
    req.ai_socktype = SOCK_STREAM;
    req.ai_flags = AI_PASSIVE; // local address

    err=getaddrinfo(NULL, SERVER_PORT, &req, &list);
    if(err) {
        printf("Failed to get local address, error: %s\n",gai_strerror(err));
        exit(1);
    }

    sock=socket(list->ai_family,list->ai_socktype,list->ai_protocol);
    if(sock==-1) {
        perror("Failed to open socket");
        freeaddrinfo(list);
        exit(1);
    }

    if(bind(sock,(struct sockaddr *)list->ai_addr, list->ai_addrlen)==-1) {
        perror("Bind failed");
        close(sock);
        freeaddrinfo(list);
        exit(1);
    }
    freeaddrinfo(list);

    listen(sock,SOMAXCONN);
    puts("Accepting TCP connections (IPv6/IPv4). Use CTRL+C to terminate the server");
    adl=sizeof(from);

    for(;;) {
        newSock=accept(sock,(struct sockaddr *)&from,&adl);
        if(!fork()) {
            close(sock);
            getnameinfo((struct sockaddr *)&from,adl,cliIPtext,BUF_SIZE, cliPortText,BUF_SIZE,
NI_NUMERICHOST|NI_NUMERICSERV);
            printf("New connection from %s, port number %s\n", cliIPtext, cliPortText);
            do {
                sum=0;

```

```

        do {
            num=0;f=1;
            for(i=0;i<4;i++) {
                read(newSock,&bt,1);
                num=num+bt*f; f=256*f;
            }
            sum=sum+num;
        } while(num);
        printf("Connection %s, port number %s received %ld\n", cliIPtext, cliPortText,
sum);

        n=sum;
        for(i=0;i<4;i++) {
            bt=n%256;
            write(newSock,&bt,1);
            n=n/256;
        }
        } while(sum);
        close(newSock);
        printf("Connection %s, port number %s closed\n", cliIPtext, cliPortText);
        exit(0);
    }
    close(newSock);
}
close(sock);
return 0;
}

```

After opening the SOCK_STREAM (TCP) socket, it's bound to the local address (including the fixed port number), and the pending connections request queue size is defined (listen). The main (infinite) loop calls **accept()**, when it unblocks, it returns a new socket connected to the client (**newSock**), **fork()** is then used to create a child process to handle the client's requests on **newSock**, meanwhile the parent process calls **accept()** again for additional clients.

3. [Lab02.ex03] Implementing a Key-Value store

Using the KV-store implementation with UDP from last practical class proceed to change it to use the TCP protocol.

3.1. [Lab02.ex03] LLM challenge

Chat and ask to change your program as expected.

What are the main changes that were proposed? Is it correct?

Put here the answer: